

CE221 - C++ Programming Assignment 1 - 24/25

Dr Michael Barros (m.barros@essex.ac.uk)

Deadline: 13:59:59, Monday 25th November 2024

Notice

You should refer to sections 5 and 7 of the Undergraduate Students' Handbook for details of the University policy regarding late submission and plagiarism; the work handed in must be entirely your own. Your programs will be checked by an intelligent plagiarism detection system that looks for similarities between the submitted programs. (Since some of the code has been supplied on Moodle, there will inevitably be similarities, so I will examine any submissions flagged by the checker to determine whether the similarities that have been detected are reasonable.)

This assignment comprises two exercises. The files for use in this assignment are available in Moodle as a zip file. Submission of this assignment will be via FASER.

Exercise 1: Library Management System in C++

In this assignment, you need to write a basic library management system in C++ with the following objectives:

1. Create new members and store them in a file. This must include their information as well.
2. Let the member log into the system and see their borrowed books.
3. Store and show their three most recently borrowed books.

Instructions

Create a header file for a class `Member` that includes the following private members: `name`, `age`, `membershipID`, `password`, and `borrowedBooks` (in the plural sense). You must think of data types for those variables that take the **LEAST** amount of space in memory. Now, create the implementation of the class. The main requirement is that the you have the definition of the class in a `.h` file and the implementation must be in a separate `.cpp` file. Now, create the main file. Make sure that all members and their information are stored in a `.txt` file. You will ask the users the following option:

Please select a number for the option of:

- 1- Login
- 2- Register
- 3- Borrow a book
- 4- Exit

Option:

If the member chooses option 1, you need to check the `membershipID` and `password` of the member, and only when you find a matching entry in the `.txt` file should you consider the member authenticated. After that, you must create an object of type `Member` with all the information of the member contained in that file. You must then print all the private members of that class (except `password`!) using the `<<` operator. For the list of books borrowed, you must print all the information for each book that you have also in the `.txt`. For example, an output after the member authenticates could be:

```
Welcome John Smith!
Your age is 30
Your membership ID is M12345.
Your borrowed books are:
- "The Hobbit" by J.R.R. Tolkien
- "1984" by George Orwell
- "To Kill a Mockingbird" by Harper Lee
```

For option 2 in the menu, you must ask the member for their details and then create a new entry in the `.txt` file. You must also ask how many books they are going to borrow initially. Collect the information from the members and store them in the file. For option 3, you need to register a new borrowed book entry for a logged user in the file. You are free to define how you interact with the member to ask all the information you need to register the new borrowed book. Of course, that includes the title and author of the book. For option 4, exit the program cleanly.

Challenge

ALL variables in the main should be dynamically allocated. The main requirement is that you don't get any segmentation fault during the execution of the program. I don't accept tricks including calling a new function from the main that is in the heap, etc.

Exercise 2: Text File Analysis Program in C++

In this assignment, you must write a program that analyses a large library catalog file. The main objectives are:

1. Search for five authors in a catalog file. These authors are given by a user.
2. Store the output produced in another text file. Plus, display the occurrence of these authors in a bar graph. The output should be formatted as follows (for the application output only):

Searched authors: Tolkien, Orwell, Austen, Rowling, Hemingway.

Author Occurrence (**not representative*):

```
Tolkien    ===== 6 (10%)
Orwell     ===== 4 (2%)
Austen     ===== 2 (1%)
Rowling    ===== 5 (6%)
Hemingway  ===== 2 (1%)
```

Instructions

The program should ask the user for the names of the catalog file to be analyzed, the file containing the search authors, and the output file. The program should terminate cleanly with an appropriate error message if any of the files cannot be opened.

Your program must contain a class called `ReadCatalog` and a function for generation of the bar graph (which should send its output to both `cout` and the output file).

The header file for the `ReadCatalog` class and a partial implementation are provided in the `ex2` folder in the zip file.

A sample catalog file is provided to allow you to test your program. A separate (unseen) author list will be used to test your submitted work; this list will not contain more than 5 authors and it will have a newline character at the end like the author list file provided.

For this assignment, an “author” should be regarded as a sequence of two words after a “by”. You must use the function `extractAuthor`, defined in the header file `.h`, return a string containing the author from the file line. You are allowed to remove punctuations from the names or add lower/upper case mechanisms if makes it easier for you.

It is recommended that you input the list of authors into an array of strings before processing the catalog file. (If `s` is a string variable, a statement such as `file >> s;` inputs the next word from the file into `s`, skipping any white space.)

Bar Charts

Bar charts show visually the frequencies of certain items, the widths or heights of the bars being proportional to the frequency of author occurrence. The percentage, next to the number of author occurrences, is given by the number of author occurrences divided by the total number of authors in the catalog file.

Challenge

Overload the `<<` operator of the class `ReadCatalog` to automatically provide the output just by trying to print a `ReadCatalog` type object. The output should be sorted with an order rule of your choice.

Deliverables

Submit a single zip file containing two folders: **ex1** and **ex2**. The only file formats acceptable are **.zip**, **.7z** or a **linux gzipped tar file**. If you submit a file in any other format, you will lose 10% of the marks for this assignment. The folder **ex1** should contain all the **.h** and **.cpp** files for Exercise 1, along with a **.txt** file produced as output. The folder **ex2** should contain the **ReadCatalog.h** file, **ReadCatalog.cpp**, and **Main.cpp**. **Files generated by the compiler, such as .o and .exe files, should not be submitted.**

Marking Scheme

	Criteria	Marks
Good Programming Practices	Readability	5
	Efficiency	10
	Commenting	5
	Lab Attempts	10
Exercise 1	Implementation of Member Class	10
	User Registration & Authentication	15
	Challenge	10
Exercise 2	Complete ReadCatalog Class	10
	Method to Display the Bar Graph	10
	Correct Output	5
	Challenge	10
Total		100

Detailed Marking Scheme

	Criteria for Mark Deductions	Deduction (Marks)
Good Programming Practices	Poor code readability (e.g., inconsistent indentation, messy formatting) Unclear or non-descriptive variable/function names Creating double variables/objects in memory Lack of or insufficient commenting Not attempting lab exercises or incomplete attempts Code not compiling fully Abrupt running interruption Extra code not related to the exercise Wrong format submitted	Up to -5 Up to -3 Up to -5 Up to -5 Up to -10 Up to -10 per ex Up to -10 per ex Up to -10 per ex Up to -10 per ex
Exercise 1	Missing or incorrect implementation of private members in Member class Not using memory-efficient data types for class members Implementation not separated into header and .cpp files User registration not storing data correctly in the file Authentication system fails to validate users properly Printing private members without overloading the << operator Password displayed or not securely handled Not displaying recent borrowed books correctly Not attempting or completing the challenge requirement	Up to -5 Up to -5 -2 Up to -3 Up to -7 -3 -2 Up to -3 -10
Exercise 2	ReadCatalog class incomplete or methods not functioning Not handling file input/output errors appropriately Incorrect parsing of the catalog file or author names Bar graph not displayed correctly or not matching specified format Output not sent to both console and output file as required Percentage calculations incorrect or missing Not overloading the << operator for ReadCatalog class Not attempting or completing the challenge requirement	Up to -2 Up to -5 Up to -3 Up to -7 -3 Up to -2 -3 -10
Total Possible Deductions		-100